



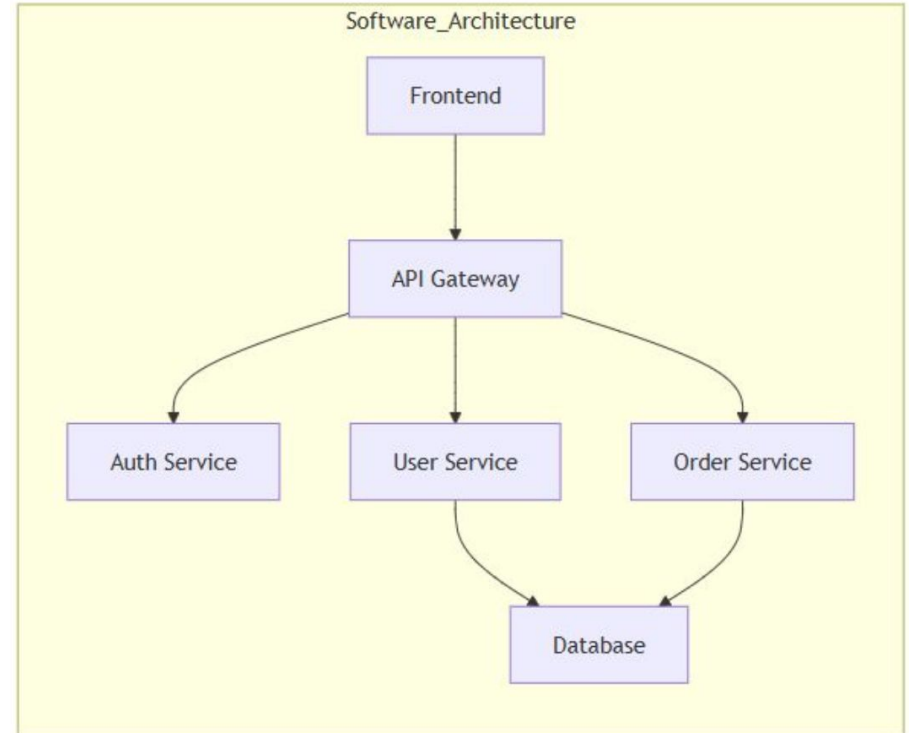
Week 3

- Software Architecture

Software Architecture Definition

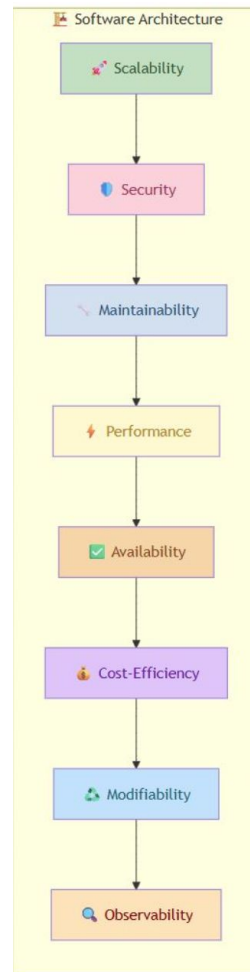
Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures.

- It involves high-level design decisions about components, their responsibilities, and their interactions.



Goals

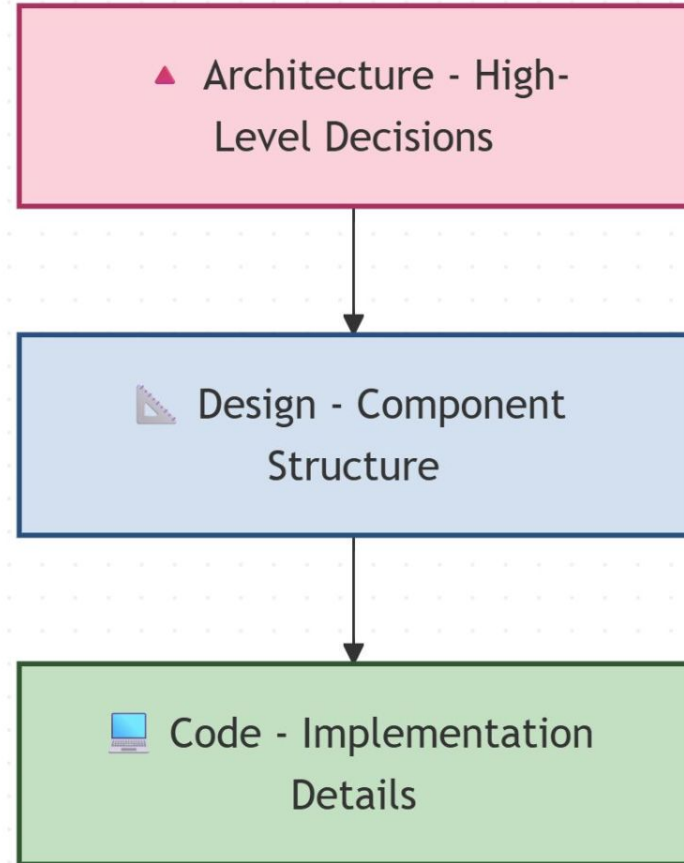
- Provide a structured solution that meets technical and business expectations.
- Support scalability, reliability, performance, security, and maintainability.
- Allow for decision-making clarity at high abstraction levels.





Arch vs Eng vs Design

- Architecture = High-level structure & constraints
- Design = Mid-level patterns & interactions
- Code = Detailed implementation

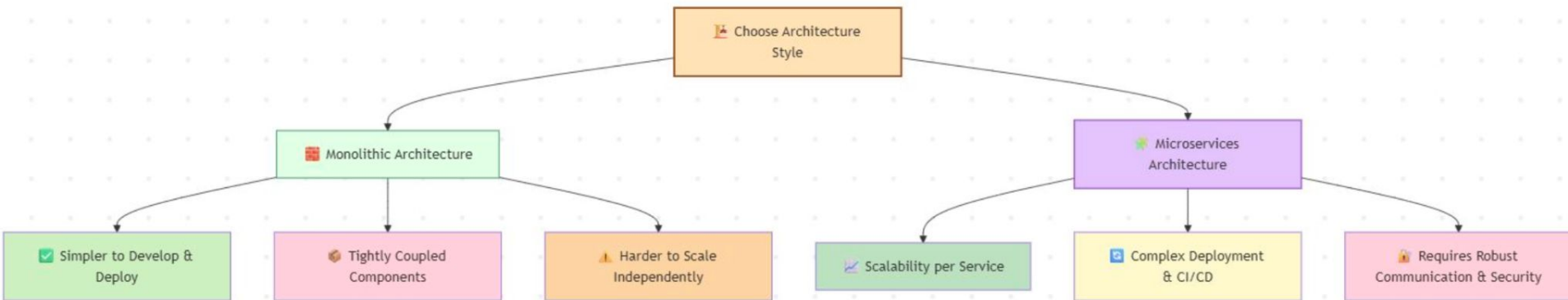


Structure & Design

Not just what components exist - but why they exist, and how they interact

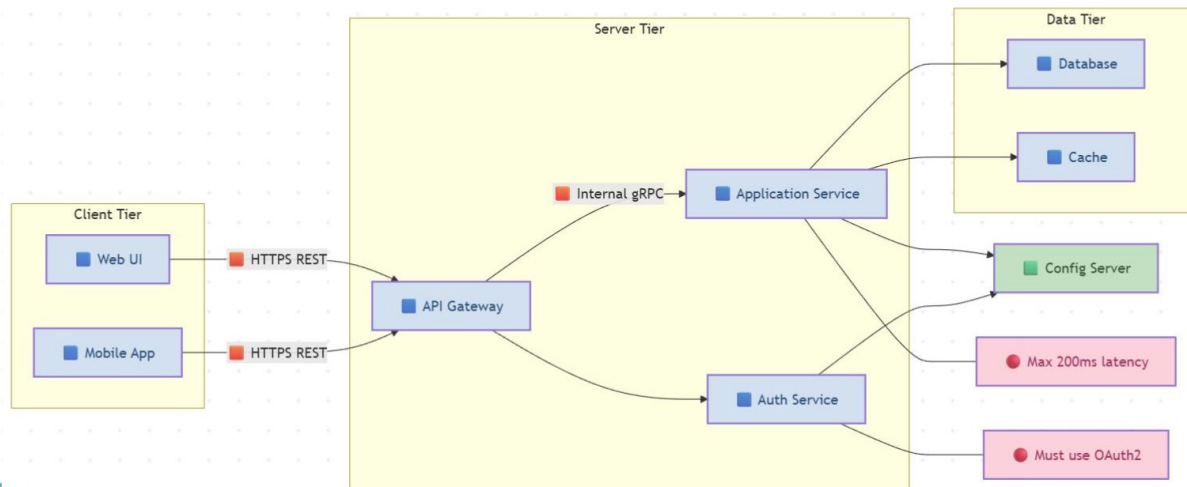
Key concept: Architectural Decisions

- What to centralize vs. decentralize
- What tech to use
- What to make resilient or scalable



Key Elements of Software Architecture

- Components (e.g., services, modules)
- Connectors (e.g., APIs, message queues)
- Configuration (deployment, runtime behavior)
- Constraints (performance, cost, latency)



Impact on Quality

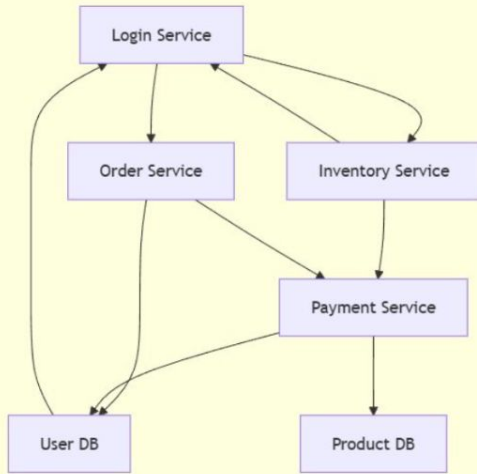
Architecture influences:

- Performance: latency, throughput
- Scalability: horizontal/vertical
- Availability: failure tolerance
- Security: surface area, boundaries
- Maintainability: modularity, testability

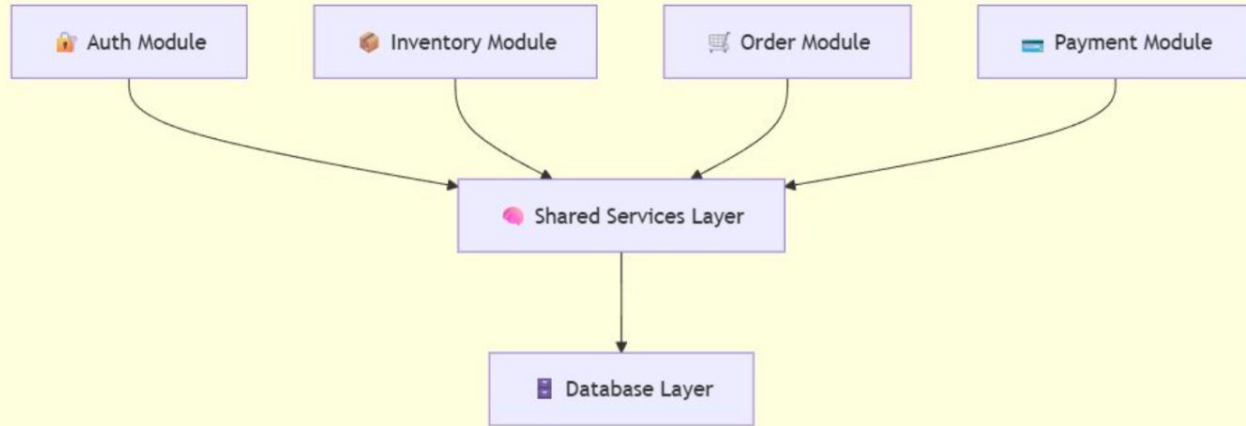


Real World Consequences

Before

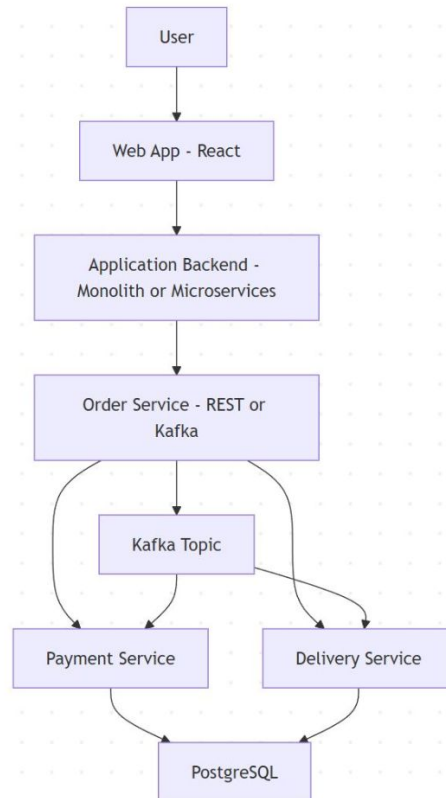


After



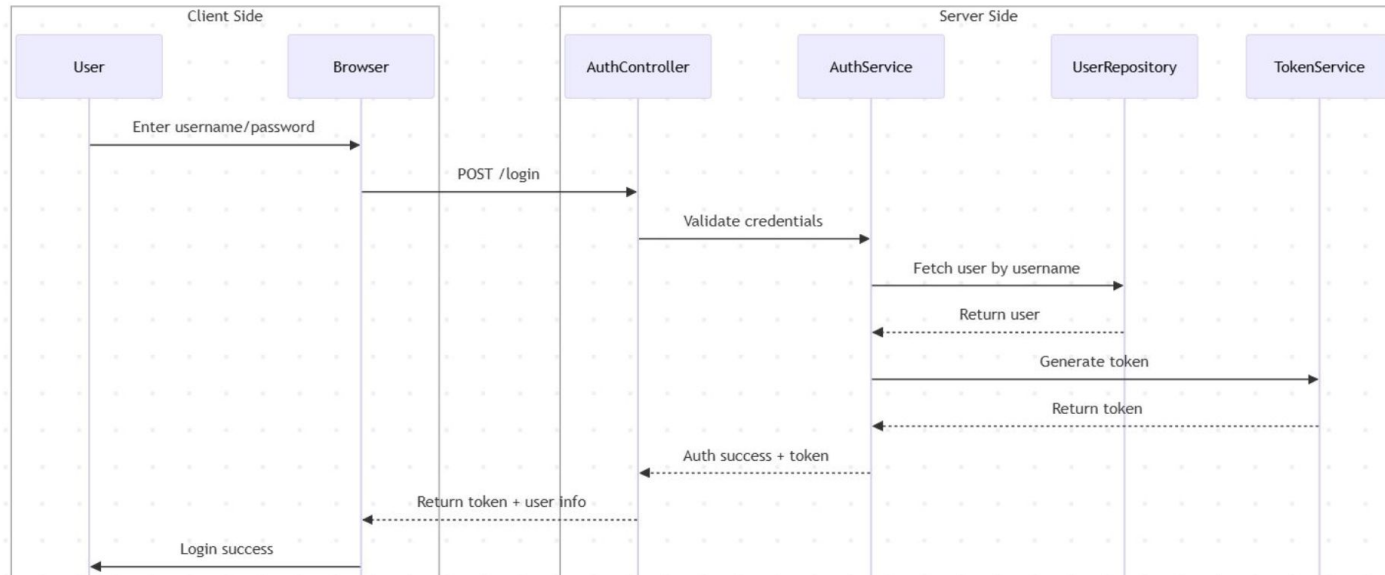
Architecture the Big Picture

- Defines what parts exist, how they interact, and why they're structured that way.
- Concerned with scalability, deployment, constraints, team boundaries



Design - Blueprint for Devs

- Focused on how individual components are implemented and interact.
- Common topics: patterns (e.g., MVC), error handling, state management.





Code, Final Layer

- This is where everything becomes real source files, functions, classes, tests.
- Reflects decisions made at the design and architectural levels

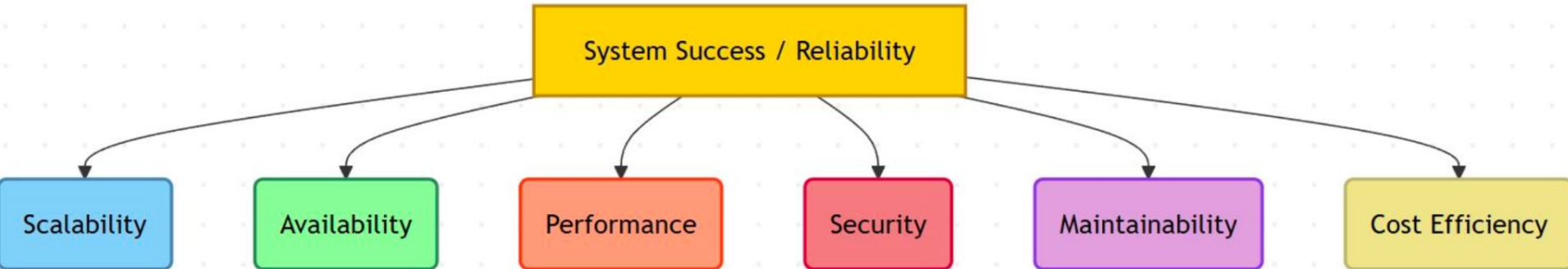
```
public class UserService
{
    private readonly IUserRepository _userRepository;
    private readonly IEmailSender _emailSender;

    public UserService(IUserRepository userRepository, IEmailSender emailSender)
    {
        _userRepository = userRepository;
        _emailSender = emailSender;
    }

    public async Task RegisterUser(UserDto dto)
    {
        var user = new User(dto.Email, dto.Password);
        await _userRepository.Save(user);
        await _emailSender.SendWelcomeEmail(user.Email);
    }
}
```

Architecture Quality Attributes

- Non-functional requirements (NFRs) that define system behavior beyond just features.
- They affect architecture decisions and user experience.
- Often have trade-offs, improving one may weaken another.



Scalability , Can System Grow

Ability to handle increased load without performance degradation.

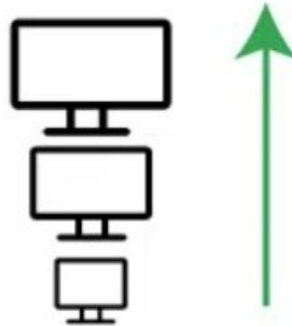
Types:

- Vertical Scaling (scale-up)
- Horizontal Scaling (scale-out)

Examples:

- Adding servers behind a load balancer
- DB sharding for distributed load

VERTICAL SCALING
Increase size of instance
(RAM, CPU etc.)



HORIZONTAL SCALING
(Add more instances)



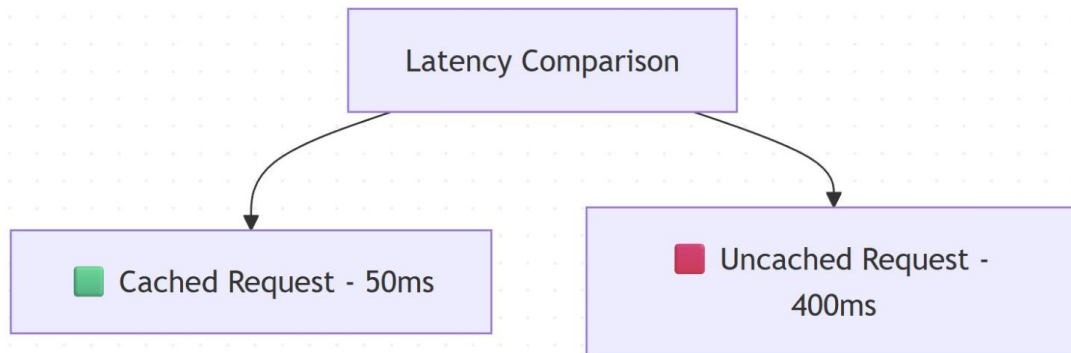
Types of Scalability

- Read Scalability – CDN, caching (Redis, Memcached)
- Write Scalability – DB partitioning/sharding, async queues
- Elastic Scalability – Auto-scaling on cloud (AWS/GCP)



Performance

- Speed at which system processes requests.
Includes:
 - Latency (response delay)
 - Throughput (requests/sec)
 - Load time, batch processing time, etc.
- Optimized via caching, compression, indexing, CDN, etc.



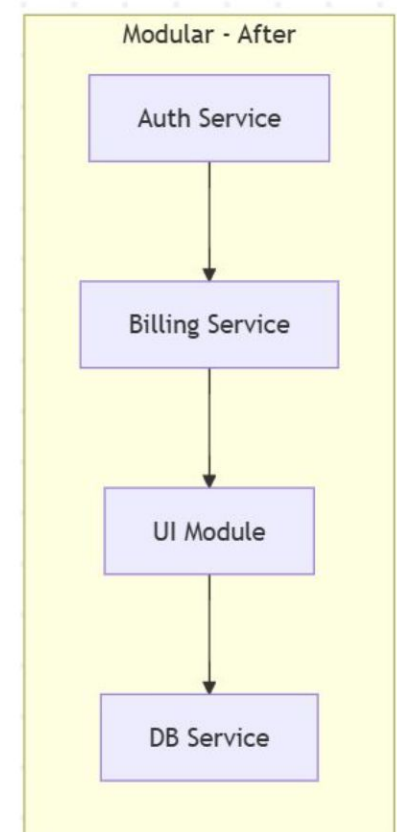
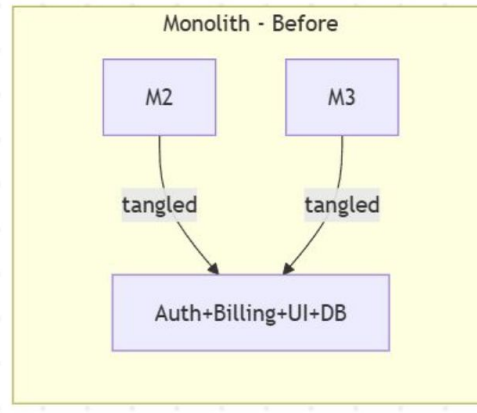


Availability

- Uptime of the system (measured in 9s: 99.9%, 99.99%)
- Downtime due to:
 - Server crashes
 - DB failures
 - Network outages
- Patterns: Redundancy, failover, heartbeat checks, retries

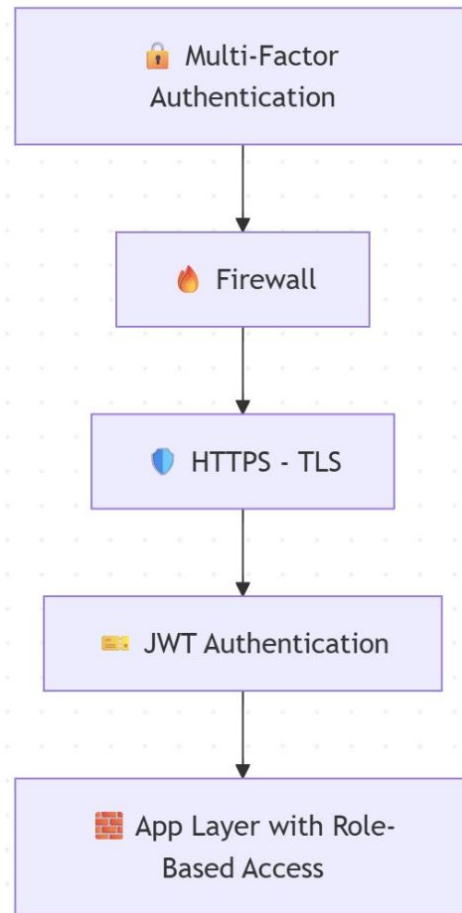
Maintainability - Can we evolve System

- Ease of modifying the system with minimal risk.
- Factors:
 - Code structure (modularity, readability)
 - Automated testing, CI/CD
 - Documentation
- Monoliths vs. Microservices trade-off



Security

- Protecting system from unauthorized access & breaches.
- Consider:
 - Authentication & Authorization
 - Data encryption (at rest/in transit)
 - Secure coding practices
 - Input validation, rate limiting
- Zero trust model & least privilege principle



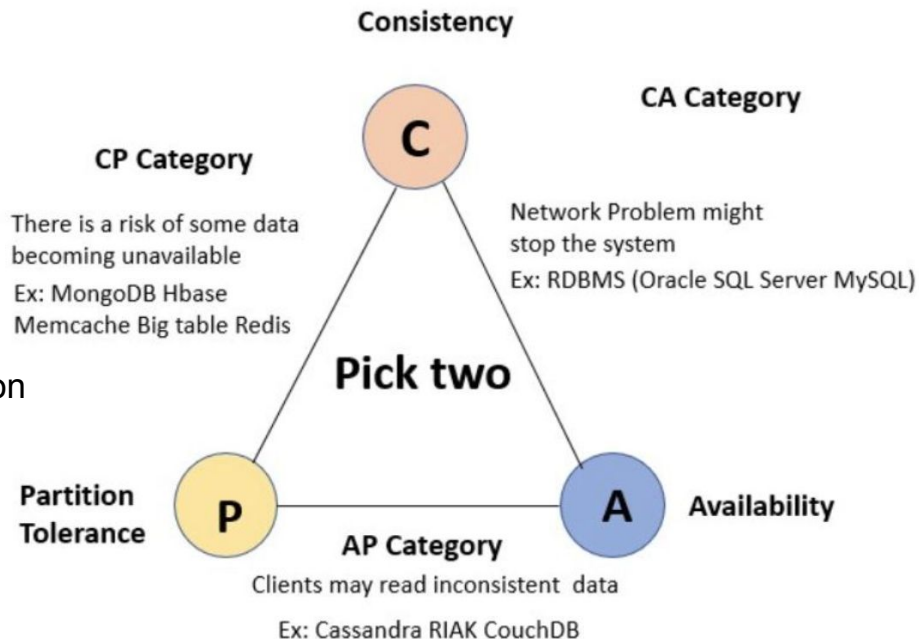
Trade Offs Between Quality Attributes

CAP Theorem:

- Consistency vs. Availability vs. Partition Tolerance

Other trade-offs:

- Performance vs. Security (e.g., encryption overhead)
- Maintainability vs. Performance (e.g., abstraction layers)





ADR - Documenting Architecture

Lightweight docs capturing important decisions

- Format: Title, Context, Decision,
- Status, Consequences
- Example decisions: use of
- PostgreSQL over MongoDB, JWT
- over Session ID

Example

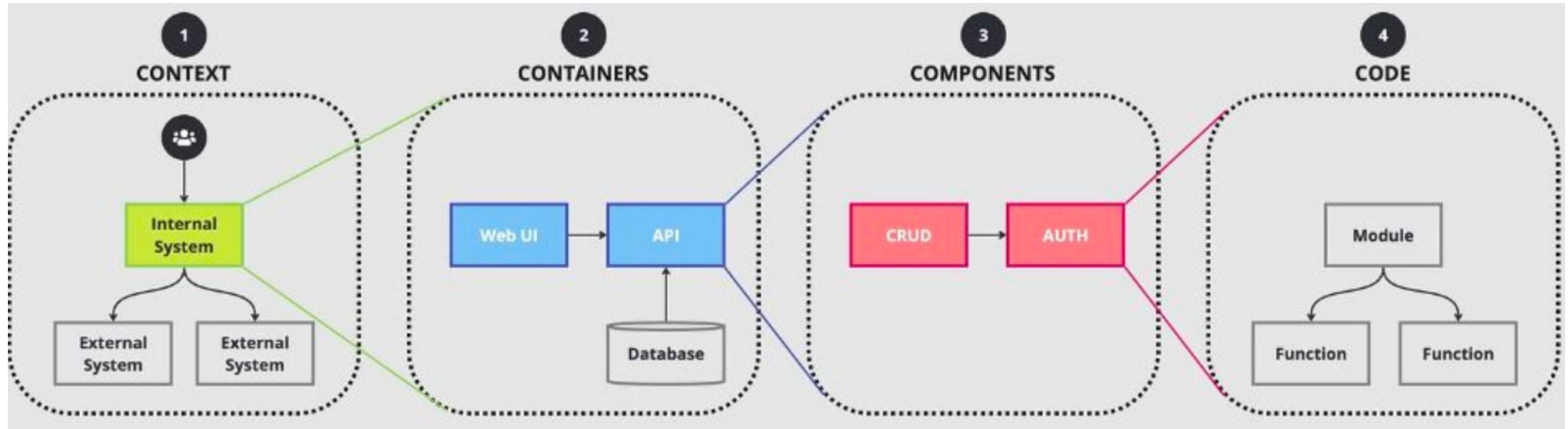
- Title: Descriptive title
- Status: Proposed, Accepted, Deprecated
- Context: What led to this decision
- Decision: What was chosen and why
- Consequences: What changes, what's impacted

Architecture Decision Record (ADR) Template

Title	Short name of the decision to be made
Date	Date when the decision was made
Context	Explanation of the problem
Decision	Clear statement of the decision made
Alternatives	Description of alternative solutions that were considered
Consequences	Implications of the decision, positive or negative
Participants	Names of team members how made the decision
Status	The current status of the decision, such as "proposed," "accepted," "rejected," or "superseded."

The C4 Model Overview

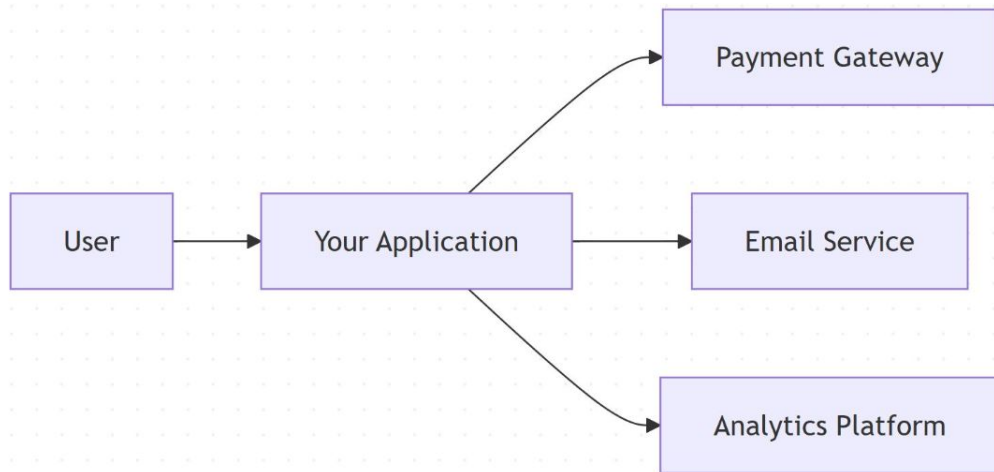
- C4 = Context, Container, Component, Code
- Helps visualize software at different levels of abstraction
- Focuses on static structure



C4 Level 1 - Context Diagram

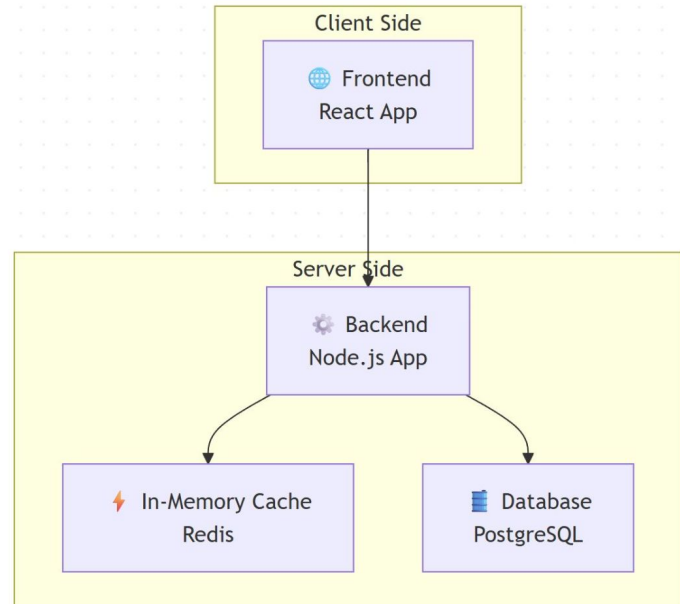
Big picture: How your system fits into the environment

Shows users, external systems, and primary interfaces



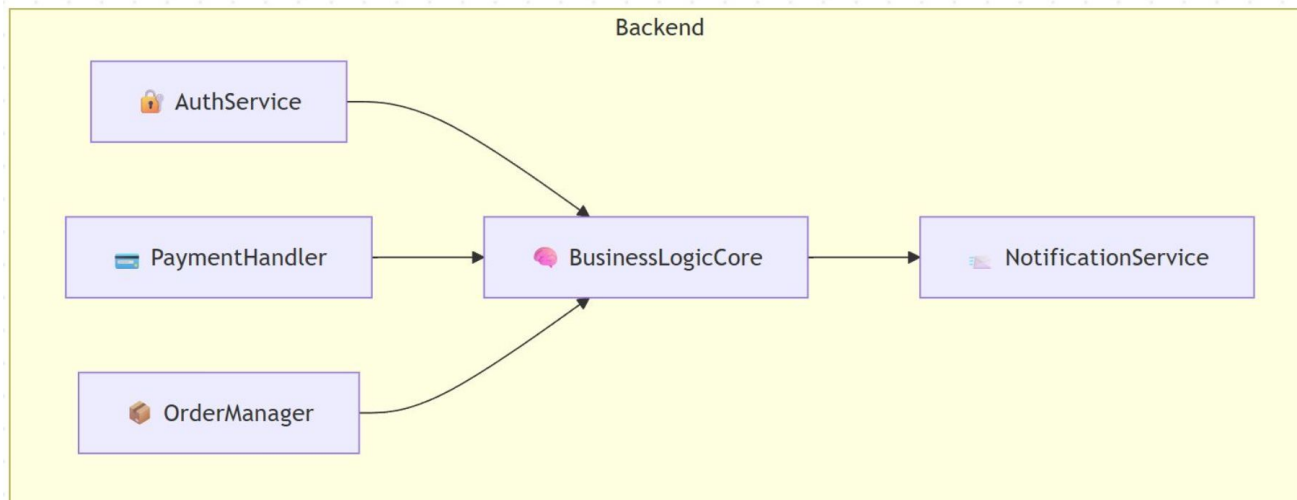
C4 level 2 - Container Diagram

- Zoom into your system
- Shows major building blocks: Web App, API, DB, Services
- Describes tech used (e.g., React frontend, Node.js backend)



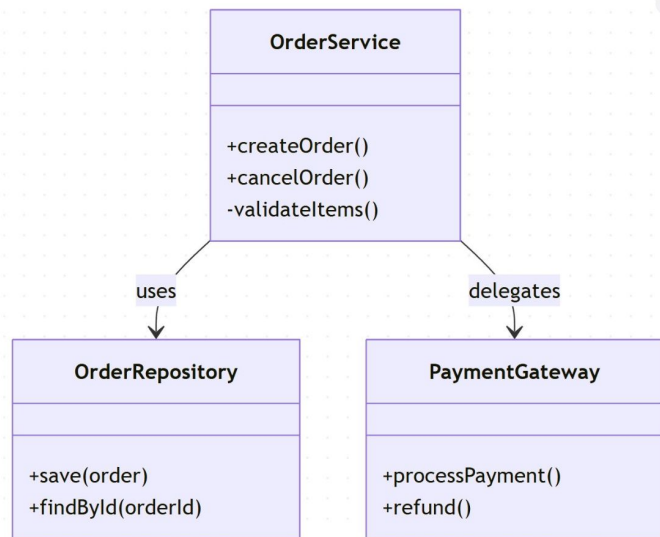
C4 level 3 - Component Diagram

- Inside a container: what components/modules exist?
- Shows how services, controllers, and libraries interact



C4 level 4 - Code (Optional)

- Lowest level: class diagram or package diagram
- Often too detailed for regular docs
- Use sparingly, mainly for teaching/onboarding

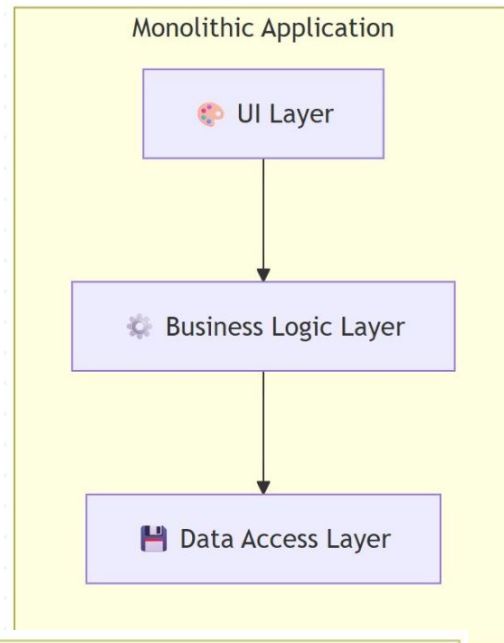


Monolithic Architecture

- A monolith is a single, self-contained application where all functionality—UI, business logic, data access, resides in one tightly integrated codebase.
- Deployed as one unit, runs as one process.

Why Learn This First?

- It's the foundation of most traditional systems.
- Many companies (especially legacy enterprises) still operate large monoliths.
- You can't truly understand microservices or modular architectures without grasping what we're evolving from.



Learning Importance

✓ Still common in legacy systems

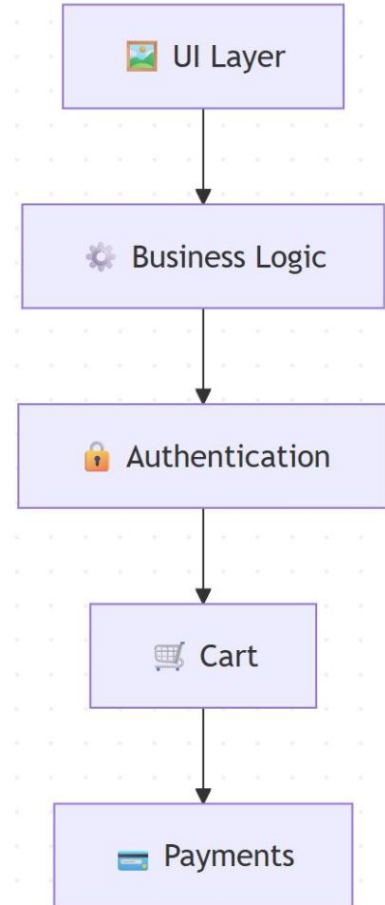
✓ Easier for MVP/startups

✓ Simpler deployment process

✓ Foundation for understanding microservices

Monolithic Characteristics

- Tightly coupled components
- Shared memory and codebase
- Deployed as one unit
- Easy to develop early on
- Hard to scale selectively

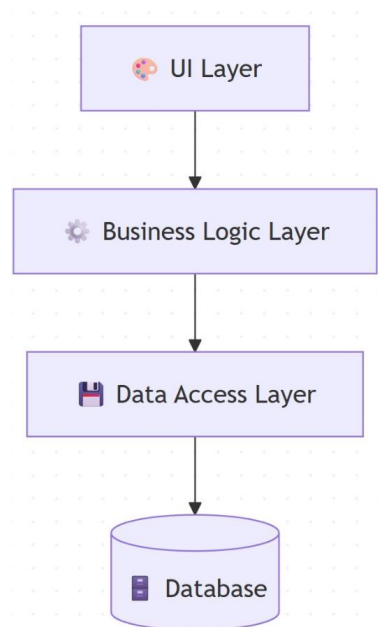


Intro to layered Architecture

- Layered Architecture = logical separation of concerns

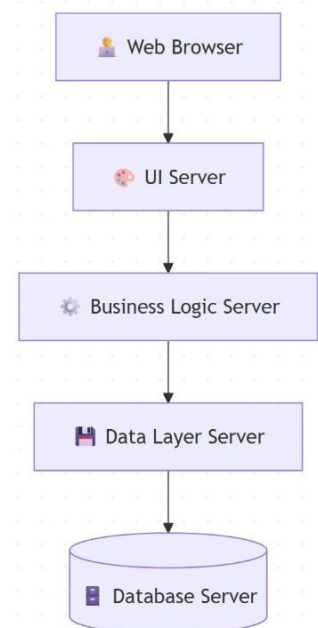
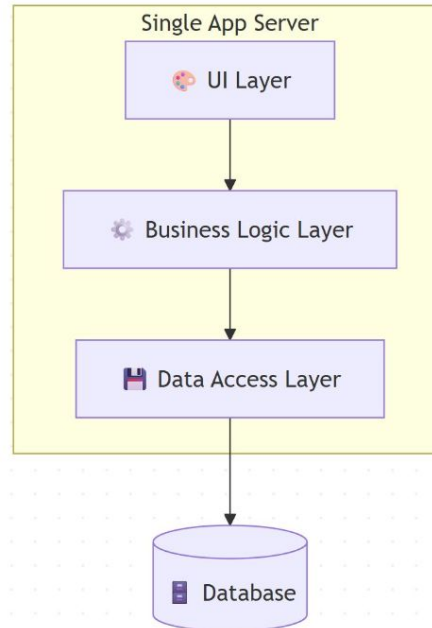
Typical layers:

- Presentation/UI
 - Business Logic
 - Data Access
-
- Often used inside monolithic applications.



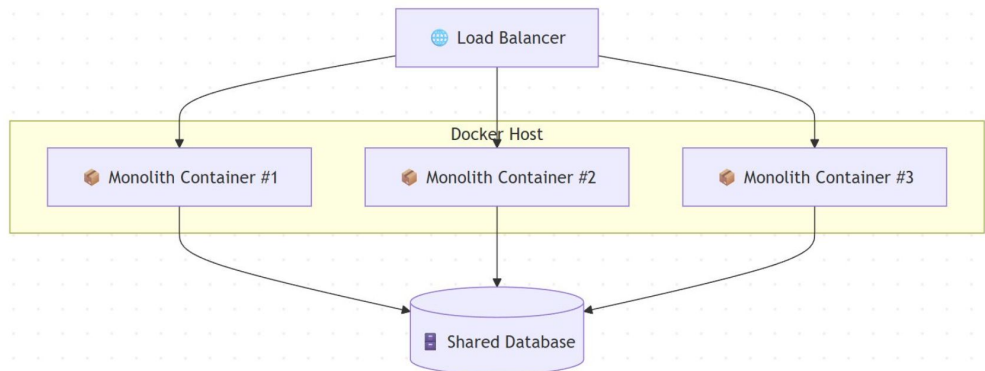
3 Tier vs N Tier

- 3-tier = Presentation, Logic, Data
- N-tier = Same logic but layers may be physically deployed separately
- Adds scalability and flexibility



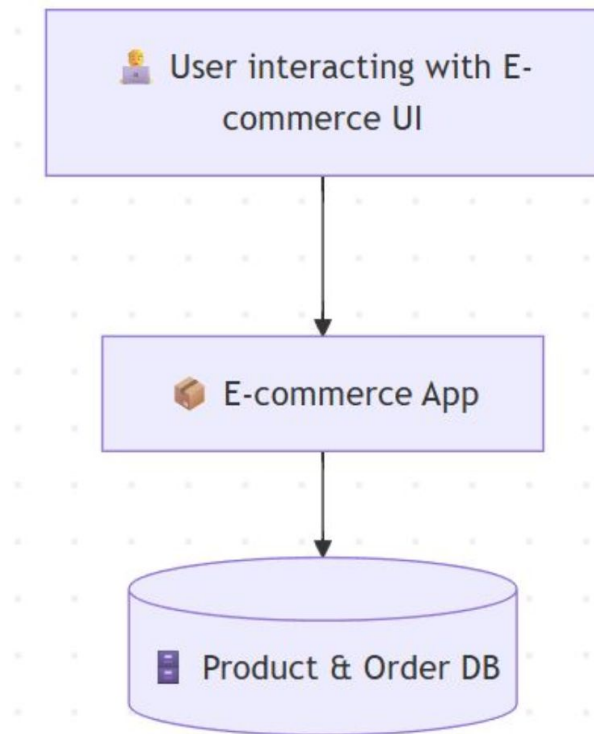
Deployment Model for monolith

- Deployed as a single artifact (WAR, EAR, binary, container)
- Scaling = duplicating the whole app (horizontal scaling)
- Small changes require full redeploy



Real World Example

- Example: A traditional e-commerce website
 - One application handling everything from login to checkout to payment
- Early versions of Amazon, eBay were monoliths





Benefits vs Limitation

Benefits

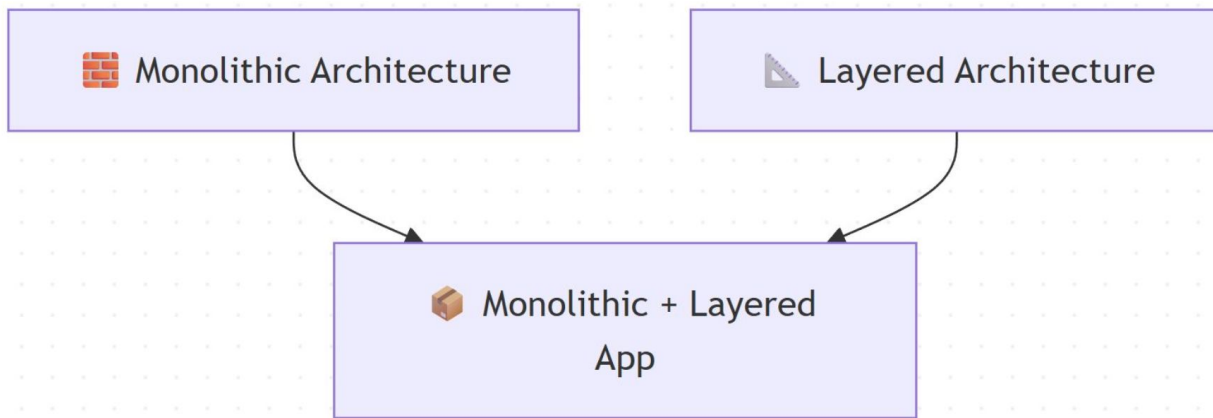
- ✓ Easy to develop initially
- ✓ Simple to deploy
- ✓ Easy testing (single app context)
- ✓ Performance (no network overhead between modules)

Limitation

- ✗ Hard to scale specific features
- ✗ Tight coupling = risk in change
- ✗ Long build and deployment times
- ✗ Difficult to adopt new tech stack per module
- ✗ Barrier to team scaling

Monolith vs Layered Architecture

- Layered Architecture can exist within monolith
- Monolith = Deployment strategy
- Layered = Design strategy
- Both can coexist





Thank You